Agent Evaluation Checklist

(1/30/2026 In review)

How to Use This Guide

Agent evaluation (eval) should be an iterative process seeding from the agent envision/design phase, continuing all the way through agent deployment and regression detection. This template provides the essential elements in building an agent evaluation (Eval) test sets and how to implement and iterate, through a four-stage structure throughout an agent's lifecycle.

- Stage 1 – Build foundational eval test sets
- Stage 2 – Establish baseline, iterate and refine test set & agent instruction
- Stage 3 – Systematic expansion (regression, variation, diagnostic, edge cases)
- Stage 4 – Continuous quality improvement eval operation

Stage 1 – Foundational Eval Sets Building

Goal: Create and run a foundational eval test set that assesses agent's core scenarios.

An eval **test set** is a group of **test cases**. A test case is an individual prompt-and-response pair to evaluate an agent's answer to a specific question. It includes a ***test prompt*** and an optional ***expected response (assertion)*** directly manifesting from the agent instruction requirement. A test case should also specify the ***acceptance criteria*** and test method to evaluate the quality.

Agent Scenario	**Test prompt** *Sample question prompt to an agent*	**Expected Response**	**Acceptance Criteria** *Define what a successful response looks like. What passes and what not.*
Agent should answer policy content based on the policy knowledge article.	*"How many sick leave days does an employee get?"*	*"30 days. <citation>"*	○ *The response must contain the exact text from the policy knowledge -> text match.* ○ *The response must include a citation. ->?*
Agent should not answer questions beyond the policy knowledge article. Direct answers from HR human support.	*"How many sick leave days does an employee get?"*	*"The policy document does not specify the sick leave days. Consult HR on your sick leave policy."*	○ *Response to forbidden case must route to human HR support.*

Test Case template

☐ Agent Scenario _____

☐ Test prompt _____

☐ Expected response _____

☐ Acceptance criteria _____

📋 **Tip**

1. **Agent Scenario**
   A foundational test set should include test cases that cover the agent's key scenarios or use cases. Use the agent scenario as guidance, focus on what the agent is intended to handle or avoid. This process helps you compile a targeted list of test prompts, and should be closely coordinated with the development of agent instructions.

2. **Acceptance Criteria**
   Clearly define what constitutes success. This can be challenging at first, so consider refining your criteria through iteration. Run the test prompt, review the response, and evaluate its quality by asking: *Does it answer the main question? Does it use the correct information? Is the tone and style appropriate? Does it respect sharing permissions?* Your insights from these questions will help you establish acceptance criteria and, if needed, an expected response.

3. **Test Set**
   How many test cases are enough? There is no universal answer, but one test prompt for a key scenario is a good start. Start with small. Iterate and refine.

Stage 2 – Establish Baseline and Improve

Goal: Run evaluations, establish baseline metrics to benchmark and improve.

Eval can be performed manually or with specialized tools. For manual evaluation, send the test prompt to the agent, review the response, use human judgment to determine if it meets the acceptance criteria, and record the result. Microsoft offers tools for agent evaluation, including the Copilot Studio agent evaluation feature (currently in public preview) and an upcoming pro-code M365 Copilot Agent Evaluation tool for M365 agents.

Establish Baseline

- Run the foundational test set against the agent
- Documented pass/fail for each test case
- Calculated overall pass rate: _____%
- Recorded agent version and baseline date: _____

Root Cause Analysis and Iteration

Review the Eval results to identify false positives and true negatives for further analysis. A false positive occurs when an answer is marked as passing but should fail based on human judgement. A true negative is an answer correctly identified as a failure. Assess these failed cases from two angles:

1. Test case issue - Are the test prompt/expected answer/acceptance criteria leading to the failure?
2. Agent design issue - Does the failure point to unclearness in agent instruction, flaws in knowledge/tool configuration?

Identify root cause and improve by either refining the test case or improving agent design.

> 📋 **Tip**
>
> 1. **Eval Passing Score**
> AI agents can produce varying responses to the same prompt due to their probabilistic nature. This variability may cause answers to pass or fail based on how strict the acceptance criteria are. To ensure reliable evaluation, run each test set multiple times and calculate the average success rate. Aim for a realistic pass rate in the high 80s or 90s, depending on your business needs.

Stage 3 – Systematic Expansion

Goal: Build comprehensive eval suites on different agent quality category.

Stages 1 and 2 established the foundational test set for the agent's primary use cases. Next, broaden your evaluation by creating test sets that assess various agent quality categories. The following list suggests categories that address different aspects of quality.

Quality Category	Goal
Foundational core	The 'must pass' set. It gauges the essential response quality at deployment and performs regression detection during operation.
Agent Robustness	One AI agent's core value over traditional software is its robustness in handling different use cases. This can include: ○ *How does the agent respond to the same question phrased in different terms?* ○ *How does the agent handle rich context provided in the prompt?* ○ *How to measure multi-intent in a single prompt?* ○ *Can agent answer user-specific requests correctly?* Agent should handle the use case variance with grace and can be evaluated with dedicated test cases.
Architecture Test	Evaluate the agent's functional performance. Dimensions can include: ○ Tool call, action ○ Knowledge retrieval/ Citation behavior ○ Routing logic ○ Integrating handoffs
Edge Cases	How the agent should handle edge case with guardrails. ○ Boundary conditions ○ Not allowed / out of scope behaviors

💡 **Category Purpose Reference**

Core fails → Something broke. Investigate recent changes.

Robustness fail → Agent is too strict. May be overfitted to specific phrasings.

Architecture fails → Specific component or workflow needs debug.

Edge Cases fail → Guardrails need improvement. Strengthen boundaries.

Stage 4 – Continuous Quality Improvement Eval Operation

Goal: Establish continuous eval monitoring to maintain agent quality during operation.

Once an agent is deployed to production, it enters a stable phase. To maintain quality and quickly detect regressions or issues from product changes (e.g, model upgrade, knowledge system updates) or evolving use cases, set up ongoing eval operation. Set up regular eval runs or triggered by specific events for quality assurance.

- Set up **regular Eval** maintenance cadence.
- Suggested full suite eval **triggers**
    - Model change
    - Major knowledge setup update
    - New tool/connector integrations
    - Production incident

💡 **Success Indicator**
You've operationalized successfully when you can answer stakeholder concerns with specifics:
Instead of: "The agent seems okay."
You say: "Policy Compliance is at 98%, but Personalization dropped to 87%—specifically, tenure-based policies aren't being applied. We've identified the root cause and are iterating."